Advanced Algorithms: Homework 5 Solutions

Due on Mar. 06, 2024 at 11:59pm EST

Professor Dana Randall Spring 2024

As stated in the syllabus, unauthorized use of previous semester course materials is strictly prohibited in this course.

Exercise 1

Let us roll a fair 6-sided die n times. Give the best upper bound you can find on the probability that the sum of the rolls is at least $5n$.

Solution

The mean of a single die roll (1,2,3,4,5,6) is

$$\mathrm{E}[X] = \sum_{i=1}^{6} i \cdot \Pr[X = i] = \sum_{i=1}^{6} i \cdot 1/6 = 7/2.$$

The variance is

$$\mathrm{var}(X) = \mathrm{E}\left[X^2\right] - \mathrm{E}[X]^2 = \frac{1}{6}(1^2 + 2^2 + 3^2 + 4^2 + 5^2 + 6^2) - (7/2)^2 = \frac{35}{12}.$$

By linearity of expectation, the mean of n die rolls is $7n/2$. Because the rolls are independent, the variance is $\frac{35n}{12}$. So by Chebychev,

$$\Pr(X > 5n) \leq \Pr(|X - 7n/2|) > 3n/2) \leq \frac{\frac{35n}{12}}{(3n/2)^2} = \frac{35}{27n}.$$

Markov gives $(7n/2)/5n = 7/10$. The Markov's inequality bound is better when $n = 1$, then Chebychev's inequality is better when $n \geq 2$.

The Chernoff bounds given in class are not applicable, since those only apply to independent $0, 1$ random variables. You would have to prove your own version of Chernoff bounds for die rolls.

Exercise 2

We are trying to predict the outcome of an election. Suppose that there are two candidates and the population of N people has exactly N/3 supporters of candidate 1 and the rest support the second candidate. If we take a uniform sample of n people from this population to poll (with replacement), what is the probability that the majority of this n prefer candidate 1? How large should n be to guarantee that the probability that candidate 2 wins (in our sample) is at least 4/5 according to Markov's inequality? Can you get a better bound on n using Chebyshev's inequality? Chernoff bounds?

Solution

Let X be the number of people who vote for candidate 1. Each person sampled has a 1/3 chance of voting for candidate 1, independently of other people sampled, thus this is a binomial distribution, $Bin(n, 1/3)$. Immediately, we have:

$$\Pr(X > n/2) = \sum_{i > n/2} i \cdot \Pr(X = i) = \sum_{i > n/2} \binom{n}{i} (1/3)^i (2/3)^{n-i}.$$

However, actually calculating how large n needs to be so that this is at most 1/5 is very challenging, which is where the bounds we've been studying are useful. Note $\mathrm{E}[X] = n/3$, and $\mathrm{var}(X) = 2n/9$, both taken from the binomial distribution.

By Markov's inequality, candidate 2 wins in our sample if $X < n/2$:

$$\Pr(X > n/2) \leq (n/3)/(n/2) = 2/3.$$

This cannot be made lower than 1/5

By Chebychev's inequality,

$$\Pr(X > n/2) \leq \Pr(|X - n/3| > n/6) \leq (2n/9)/(n/6)^2 = 8/n.$$

To get this $\leq 1/5$, we need $n \geq 40$

By Chernoff Bounds,

$$\Pr(X > n/2) = \Pr(X > (1 + 1/2)(n/3)) \leq e^{\frac{-(1/2)^2 \cdot n/3}{3}} = e^{\frac{n}{36}}.$$

Using this, we see that n needs to be at least 58.

Exercise 3

Consider the following sorting algorithm for n real numbers chosen independently and uniformly from the range $[0, 1)$. Place each number a_i in one of the buckets $B_1, B_2, \ldots B_n$ as follows: a_i goes in bucket B_j if $a_i \in \left[\frac{j-1}{n}, \frac{j}{n}\right)$. Sort each bucket, (using your favorite sorting algorithm) and output the sorted list.

(a) For any integer $0 < M < n$, show that the probability that bucket B_i has at least M entries is at most $1/M$.

(b) Using Chebychev's inequality, give an upper bound on the probability that there exists some bucket with at least M entries.

(c) Can you do better using Chernoff bounds? Give a bound on the probability that there exists some bucket with at least M entries.

Note: Check the restrictions on the Chernoff bound we presented in the notes carefully. Now observe that $\ln(1 + \delta) > \frac{2\delta}{2+\delta}$ for all $\underline{\delta > 0}$. This implies that

$$\delta - (1 + \delta)\ln(1 + \delta) \leq \frac{-\delta^2}{2 + \delta}.$$

Solution

This problem was a bit of a trick question - consequently we were generous with the grading.

The over arching idea was: if each bucket has less than a constant M number of elements, then each bucket could be sorted in constant time, and we would be left with a $O(n)$ algorithm for sorting We use various probability bounds to calculate the probability that each bucket has less than M elements.

The "trick" part was this: No matter how you did the bounds, M would have to be a growing function of n to get a probability less than 1. This was a little vague in the problem statements, so we gave credit based on correctly doing the bounds.

(a) A particular bucket has $Bin(n, 1/n)$ distribution, which has mean 1. By Markov's inequality, $\Pr(B_j \geq M) \leq 1/M$.

(b) A $Bin(n, 1/n)$ distribution has variance $\frac{n-1}{n}$. You also could've calculate this directly using indicator variables (as in problem 2). Thus by Chebychev's Inequality,

$$\Pr(B_j \geq M) \leq \Pr(|B_j - 1| \geq M - 1) \leq \frac{n - 1}{n(M - 1)^2}.$$

Taking a union bound over all buckets, the probability that *any* bucket has more than M elements is $\leq \frac{n-1}{(M-1)^2}$. For this to be < 1, M would have to be $> \sqrt{n} + 1$. (Which means we do not actually sort each bucket in constant time.)

(c) In the notes, we obtained the Chernoff bound given in class by first deriving

$$\Pr\left(X > (1+\delta)\mu\right) \leq e^{(\delta-(1+\delta)\ln(1+\delta))\mu}$$

and then using the fact that

$$(1+\delta)\ln(1+\delta) > \delta + \frac{\delta^2}{2} - \frac{\delta^3}{6} \geq \delta - \frac{\delta^2}{3} \tag{1}$$

.

to get

$$\Pr\left(X > (1+\delta)\mu\right) \leq e^{\frac{-\delta^2\mu}{3}} \quad (0 < \delta < 1).$$

But, we need a bound that works for all $\delta > 0$. Using the hint, we know that

$$(1+\delta)\ln(1+\delta) \geq \delta - \frac{-\delta^2}{2+\delta}.$$

Substituting this bound in place of (1), we get a alternate version of Chernoff bounds that worked for all $\delta > 0$:

$$\Pr(X > (1+\delta)\mu) \leq e^{\frac{-\delta^2\mu}{2+\delta}}.$$

We will use this second bound. We first calculate the probability that any given bin has at least M elements in it, where μ is 1 and δ is $M-2$:

$$\Pr(B_j \geq M) = \Pr(B_j > M-1) = \Pr(B_J > (1+(M-2))1) \leq e^{-(M-2)^2}M.$$

After the union bound, this is $ne^{\frac{-(M-2)^2}{M}}$. Here, we'd need $M \approx \log(n)$ to get a probability < 1, which is much better, but still not constant time per bucket.

Exercise 4

We are given a set of points $\{p_i\}$ on the plane, and we are interested in finding the pair of points that are the farthest apart (called the *diameter*). There is an obvious $O(n^2)$ algorithm, but we want an $O(n \log n)$ algorithm. Prove the following statements and then use them to construct a fast algorithm.

(a) Prove that the pair of points that are farthest apart are both on the convex hull.

Now assume we are looking for the furthest pair of points on a convex polygon: $p_1, p_2, \ldots, p_n$, (given in order going around the polygon).

(b) For two points p_i, p_j, we construct the lines l through p_i and l' through p_j so that l, l' are perpendicular to $\overleftrightarrow{p_i, p_j}$. We say that p_i and p_j are *antipodal* if these lines l, l' both do not pass through the convex hull. Show that the pair of points that are the largest distance apart must be antipodal.

(c) Say we are given an edge of the convex hull: $e = (p_1, p_2)$. Give a method to find the point p_i farthest from the line $\overleftrightarrow{p_1, p_2}$. Your method should take $O(i)$ time or faster (not $O(n)$). (Hint: you'll need the slope of $\overleftrightarrow{p_1, p_2}$, and some basic vector knowledge.)

(d) Consider adjacent points p_n, p_1, p_2, and say that the point farthest from line $\overleftrightarrow{p_n, p_1}$ is p_i, and the point farthest from line $\overleftrightarrow{p_1, p_2}$ is p_j. Show that $j \geq i$, and the only possible points that are antipodal to p_1 are $p_i, p_{i+1}, \ldots p_j$. (Hint: Why are all other points definitely not antipodal?) Now if p_k is farthest from line $\overleftrightarrow{p_2, p_3}$, what about all possible points antipodal to p_2?

(e) Put it all together: Create an algorithm that takes $O(n \log n)$ time to find the pair of pair of points that are farthest apart, with proof of correctness. Your algorithm should take $O(n)$ steps after finding the convex hull. (Hint: Keep one pointer at p_1, and another at p_i as in part d. Alternate updates of p_i and p_1 in such a way that all antipodal pairs are considered. You can't use part c directly, but the idea should help)

Solution

Note: we will use the phrase convex hull to occasionally mean the boundary of the convex hull, especially when we use the preposition "on" (i.e. "on" the convex hull means the boundary, "in" the convex hull means the interior.)

(a) Let p_i, p_j be the points that are farthest apart. Assume one point, p_i, was not on the exterior of the convex hull. Extend the ray $\overrightarrow{p_j p_i}$ until it intersects the boundary of the convex hull at point x. If x is in the initial set of points (i.e., is a vertex of the convex hull), then this is a contradiction, as $d(p_i, p_j) < d(x, p_j)$. Alternatively, x is on segment $\overline{p_n p_m}$. Then at least one of angles $\angle p_j x p_n$ and $\angle p_j x p_m$ must be at least 90 degrees. That endpoint is farther from p_j than x (by the Law of Cosines, for instance) and thus farther from p_j than p_i, which gives a contradiction in this case too.

(b) Let p_i, p_j be the points that are farthest apart, and label the lines through these points perpendicular to $\overline{p_i p_j}$ as l_i and l_j, respectively. Suppose one of these lines, without loss of generality l_j, intersects the interior of the convex hull. Then there must be some point p_k on the other side of the line through l_j from p_i. Then again, $\angle p_i p_j p_k$ is greater than 90 degrees, and as above, p_k is farther from p_i than p_j, which is a contradiction.

(c) In constant time, we can find the distance of a point p_i from the line $\overleftrightarrow{p_1 p_2}$ in a number of ways. Find this distance for $p_3, p_4, \ldots$ one after another, storing the maximum. If we find point p_i where the distance from p_{i+1} to $\overleftrightarrow{p_1 p_2}$ is not larger than the distance from p_i to $\overleftrightarrow{p_1 p_2}$, I claim (and prove below) that p_i must be at maximum distance from $\overleftrightarrow{p_1 p_2}$, so our algorithm can return p_i without checking the remaining points - it is impossible for the true maximum to be after this local maximum. If p_i is the maximum, we stop after $O(i)$ steps. This is an example of an *output-sensitive* running time.

First, we assume all points on the convex hull are at a unique distance from $\overleftrightarrow{p_1 p_2}$ (below we discuss what to do if this is not the case). Orient the polygon so that $\overleftrightarrow{p_1, p_2}$ is horizontal and on the "bottom" (all points in the set are above it). This orientation can be used to divide the convex hull into an upper hull and a lower hull (containing p_1 and p_2).

Let a point p_i on the convex hull be a *local maximum* if both of its neighbors on the convex hull are closer to $\overleftrightarrow{p_1 p_2}$ than p_i is. We first show that such a maximum must lie on the upper hull. Suppose, for the sake of contradiction, a local maximum p_i is on the lower hull (and is not one of the endpoints of the lower hull). Then its two neighbors on the lower hull are both lower than it (because they are closer to $\overleftrightarrow{p_1 p_2}$ by assumption), as is the entire line segment between them. This contradicts the fact that p_i is on the convex hull of the points.

Now, suppose there are two non-consecutive local maxima on the upper hull. Then there must be a point between them on the upper hull that is closer to $\overleftrightarrow{p_1 p_2}$ than both. But then this point is below the segment connecting the two maxima, and so it cannot possibly be on the convex hull of the points, a contradiction. Thus, there is at most one local maximum, and so it is a global maximum.

We haven't yet considered the case where vertices on the convex hull are at the same distance from $\overleftrightarrow{p_1 p_2}$. If, during your scan, you encounter a vertex p_i where p_{i+1} is at the same distance from $\overleftrightarrow{p_1 p_2}$ as p_1, then by the same arguments as above, p_i and p_{i+1} must *both* be at maximum distance from $\overleftrightarrow{p_1 p_2}$. Either is a satisfactory answer, so your algorithm can just return p_i.

(d) Let p_i, p_j, etc. be as in the problem. First, we assume that p_n, p_1, and p_2 are not all collinear, because if they are collinear then we would have $\overleftrightarrow{p_n p_1} = \overleftrightarrow{p_1 p_2}$ and $j = i$, so we are done.

Assume, for the sake of finding a contradiction, that $j < i$. Orient the figure in the direction of $\overleftrightarrow{p_n p_1}$ so that p_i is the rightmost point, and $\overleftrightarrow{p_n p_1}$ is on the left. The upper hull of the convex body in this case is $p_1 \ldots p_i$, with p_n directly below p_1, and by assumption, p_j is on this upper hull. Consider the

line $\overleftrightarrow{p_1p_2}$. Note that because there are only right turns along the convex hull, until the total angle of these turns totals 180 degrees each vertex on the upper hull is farther away from $\overleftrightarrow{p_1p_2}$ than the one before it. These angles won't total more than 180 degrees until after you switch from the upper hull to the lower hull at p_i, so p_j falls into this realm. This means the vertex after p_j on the upper hull is farther from $\overleftrightarrow{p_1p_2}$ that p_j, a contradiction. Thus, it must be that $j \geq i$.

All lines through p_1 that do not intersect the convex hull must have slope between $\overleftrightarrow{p_np_1}$ and $\overleftrightarrow{p_1p_2}$, otherwise the line would separate those two neighbors. A necessary (but not sufficient) condition for a point p_h to be antipodal to p_1 is that there are parallel lines through p_1 and p_h, neither of which intersect the convex hull. For any $h < i$, by the arguments above, p_h is closer to both $\overleftrightarrow{p_np_1}$ and $\overleftrightarrow{p_1p_2}$ than p_i is, so any line through h with any slope ranging between the slopes of $\overleftrightarrow{p_np_1}$ and $\overleftrightarrow{p_1p_2}$ (i.e., any line through p_h parallel to a line not intersecting the convex hull through p_1) will separate p_i from p_1. Thus any such line will intersect the convex hull, meaning p_1 and p_h cannot possible be antipodal. The same argument applies for $h > j$ and the lower hull. Thus the only points that are possibly antipodal to p_1 are $p_i, p_{i+1}, \ldots p_j$.

If p_k is the point that is farthest from $\overleftrightarrow{p_2, p_3}$, then all possible points that are antipodal to p_2 fall in the range $p_j, ..., p_k$.

(e) Now we can state the final algorithm. Find the convex hull, number the points on the hull p_1 through p_n in the usual clockwise way. Keep two pointers, one at p_1 and one at p_i as defined above (it takes O(i) time to find i). Increment i until we find p_j, the point furthest from $\overleftrightarrow{p_1, p_2}$. Check all the distances p_1 with p_i through p_j, and keep track of the maximum. Now increment p_1 to p_2 and repeat, starting with your second pointer already at p_j. Both pointers increment clockwise around the circle, and at least one of the two pointers increments at each step. Thus we make a complete round of the circle in at most $2n$ steps, which takes $O(n)$ time after finding the convex hull, for a total $O(n \log n)$ algorithm.

Correctness from this algorithm follows from all of the previous parts: the two points that are farthest apart must be antipodal, and the algorithm checks all possible pairs of antipodal points, so it will find the correct solution.